# Agentic AI 1: LLM foundations and the agent loop — Advanced

Portfolio projects · lessons 46–60

Each takes 1–3 days and ends with something to show an employer.

## Project 1: Scout in production: tested, traced, budgeted and load-tested

**Goal:** Turn Scout into a pip-installable service with unit tests, cassette tests, evals, OpenTelemetry-style logs, cost caps and human approval, then prove it under 50 concurrent conversations with p50/p95/p99 numbers.

**Uses lessons:** 38, 39, 46, 47, 48, 49, 52, 54, 55

**1.** Package Scout as `scout/` with a `pyproject.toml`, a pinned model id in `config.toml`, and the `search_notes`, `calculator` and `write_file` tools behind a registry.

**2.** Write at least 12 pytest unit tests against a scripted FakeModel covering the step limit (max 8 steps), a tool that raises, parallel tool results in one message, and a per-run budget of 20,000 tokens.

**3.** Record 10 real conversations as cassettes with secrets scrubbed (authorization, x-goog-api-key) and replay them in CI so the full suite runs in under 30 seconds with no key.

**4.** Build a 25-case golden set with a scorer, run each case 5 times, and report pass@1 and pass@5 plus the per-case flake rate.

**5.** Emit one JSON line per step with gen_ai.request.model, gen_ai.usage.input_tokens, gen_ai.usage.output_tokens, run id, stop reason, tool name, latency and cost, with emails and phone numbers redacted.

**6.** Gate `write_file` behind an approve/edit/reject prompt that writes an audit record (who, when, arguments, decision) to `audit.jsonl`.

**7.** Run 50 conversations concurrently with a semaphore of 10 and a shared token bucket, and report p50/p95/p99 latency, cost per run, 429 count and retries.

**8.** Kill the load test mid-run, resume from checkpoints, and show no `write_file` call ran twice.

**Deliverable:** A GitHub repo with a green CI badge, `make test` and `make load` targets, and a README containing the eval table (pass@1, pass@5), the latency/cost table for 50 concurrent runs, one annotated trace of a failed run, and the resume-without-duplicates proof.

**Stretch:** Export the JSON-line spans to a local Jaeger or Grafana Tempo via the OpenTelemetry SDK and include a screenshot of one full run trace. · Add small-model-first routing (escalate to the bigger model only on a failed scorer check) and report cost saved against eval pass-rate lost. · Make the eval suite a required CI check that fails a pull request if pass@1 drops by more than 5 points.

## Project 2: Multi-provider LLM gateway with failover and capacity plan

**Goal:** Build the small internal gateway a platform team puts in front of every agent: one API over Gemini, a local Ollama model and a fake, with per-team keys and quotas, circuit-breaker failover, and a capacity and cost plan for 5,000 users.

**Uses lessons:** 37, 38, 50, 53, 54, 56, 57

**1.** Expose a FastAPI `/v1/chat` endpoint backed by the provider-neutral adapter, with Gemini as primary, a local Ollama model (e.g. qwen3 small) as secondary and a FakeModel for tests.

**2.** Issue per-team gateway keys read from environment or a secret file, enforce a per-team token bucket (e.g. 60 requests/min, 200,000 tokens/day) and return 429 with retry-after when exceeded.

**3.** Add a circuit breaker per provider that opens after 5 failures in 30 seconds, fails over to the secondary, and half-opens after 60 seconds.

**4.** Drive 20 requests/second for 10 minutes with a load script, inject a 3-minute primary outage (500s and timeouts from a stub server), and record error rate and p95 latency before, during and after.

**5.** Never retry 400-class or spend-cap 429 errors; prove it with a test for each error type and log request ids for every upstream call.

**6.** Write a capacity model for 5,000 users (assume 20 runs/user/day, 6 steps/run, 3,000 input and 400 output tokens/step): peak tokens per minute against provider tiers, concurrency via Little's law, the effect of a 70% cache-hit rate, 2x burst headroom and monthly cost.

**Deliverable:** A repo with the gateway, the outage drill script, and a README with a chart of error rate and latency across the outage window, the measured time-to-failover, and the capacity spreadsheet or notebook with its monthly cost estimate and assumptions.

**Stretch:** Add a shadow mode that mirrors 5% of traffic to a candidate model and logs answer diffs without returning them to callers. · Containerise the gateway with Docker Compose (gateway, Ollama, stub server) so the outage drill runs with one command.

## Project 3: Red-teaming a customer-support agent against prompt injection

**Goal:** Apply the attack catalogue to a different domain, a support-ticket agent that can look up orders, issue refunds and send email, and show with numbers how layered defences cut the attack success rate without hurting normal tickets.

**Uses lessons:** 46, 48, 49, 51, 52, 54

**1.** Build a support agent with tools `read_ticket`, `lookup_order`, `issue_refund(order_id, amount)` and `send_email(to, body)` over a fixture of 200 synthetic orders and 60 tickets.

**2.** Write 30 attacks: instructions hidden in ticket text, in order notes and in customer names, markdown-image exfiltration of other customers' emails, and confused-deputy refunds to a different order.

**3.** Write 30 benign tickets with expected actions as a utility set, and score both sets automatically from tool-call logs (not from the model's text).

**4.** Measure the baseline attack success rate and benign task success over 3 runs per case.

**5.** Add defences one at a time: untrusted content kept inside tool results, a tool permission matrix (refunds capped at the order total and $100), `send_email` limited to the ticket's own customer, and human approval for refunds over $50.

**6.** Report attack success rate, benign success rate and cost per ticket after each defence layer, and mark which attacks still get through and why (the lethal-trifecta check).

**Deliverable:** A repo with the agent, the attack and benign suites as JSONL, and a README with a defence-by-defence table of attack success, benign success and cost, plus three annotated transcripts of attacks that succeeded at baseline.

**Stretch:** Add a second, cheaper model as an input classifier that flags suspected injections, and report its precision and recall on the 60 cases. · Run the full suite on two different models and compare their baseline susceptibility.

## Project 4: Model migration canary for an invoice-extraction agent

**Goal:** Build the go/no-go pipeline a team runs before a pinned model retires: an invoice-processing agent, an eval set, and a canary report that diffs accuracy, tool-call behaviour, tokens, cost and refusals between the current and the candidate model.

**Uses lessons:** 40, 41, 47, 48, 53, 54, 59

**1.** Build an agent that reads invoice text or PDFs, calls `lookup_vendor` and `convert_currency`, and returns a Pydantic-validated record (vendor, date, line items, total, currency).

**2.** Create 40 labelled invoices including 8 hard ones (multi-currency, missing totals, handwritten-style OCR noise) and a field-level scorer.

**3.** Pin the current model id in config and run the set 3 times to establish a baseline of field accuracy, schema-valid rate, tool-call rate, input/output tokens and cost per invoice.

**4.** Run the same set on a candidate model at two reasoning-effort levels and diff every metric, including refusals and the number of validation retries.

**5.** Route 10% of a replayed day of 500 invoices to the candidate as a canary and stop automatically if field accuracy drops more than 2 points or cost per invoice rises more than 20%.

**6.** Generate a one-page Markdown migration report with a recommendation, the retirement date of the current model and a rollback plan.

**Deliverable:** A repo with the agent, the labelled set, a `make canary MODEL=<id>` target and a README with the baseline-vs-candidate table (accuracy, valid-schema rate, tool-call rate, tokens, cost, refusals) and the generated migration report.

**Stretch:** Add prompt caching of the stable system prompt and vendor table and report the measured cache-hit rate and cost change. · Send the 500-invoice backfill through a batch API and compare its cost and wall-clock time with the online run.